

Analyzing U.S. Stock Market Data with MongoDB and the Finnhub API

Introduction

In finance it is common to group stocks by industry and assume that they will perform similarly, but this can be misleading because companies in the same sector can be driven by very different fundamentals. In this analysis I used MongoDB Atlas to store and query financial data for 97 U.S. companies to see how performance varied across and within sectors during May 2026. Data came from two sources: Finnhub API for company profiles, financial metrics, earnings, and peer data, and Yahoo Finance for 21 days of daily price history (since Finnhub's price data requires a paid key). Each company was stored as a single MongoDB document with nested arrays and objects, making it a good fit for MongoDB's document model compared to a relational database where this data would need multiple tables.

Dataset and Storage Model

The MongoDB collection contains 97 documents, one per company. Each document includes top-level fields like symbol, name, country, industry, exchange, and market capitalization. Inside each document I nested the three main structures: a metrics object containing over 50 financial indicators such as PE ratio, beta, dividend yield, and 52-week high/low prices; an earnings array with the most recent quarterly earnings results including actual EPS, estimated EPS, and the reporting period; and a price_history array containing 21 daily records with open, high, low, close, and volume fields. This type of hierarchical structure allowed me to write MongoDB aggregation queries that operated on nested arrays directly. An example of this being trying to extract the first and last closing prices from price_history to calculate monthly returns or/and filtering the earnings array to find companies that beat analyst estimates.

Analysis

There were 10 MongoDB queries that covered a range of operations that included filtering, sorting, regex matching, and multi-stage aggregation pipelines. I thought that two of the query results were pretty interesting and are visualized below.

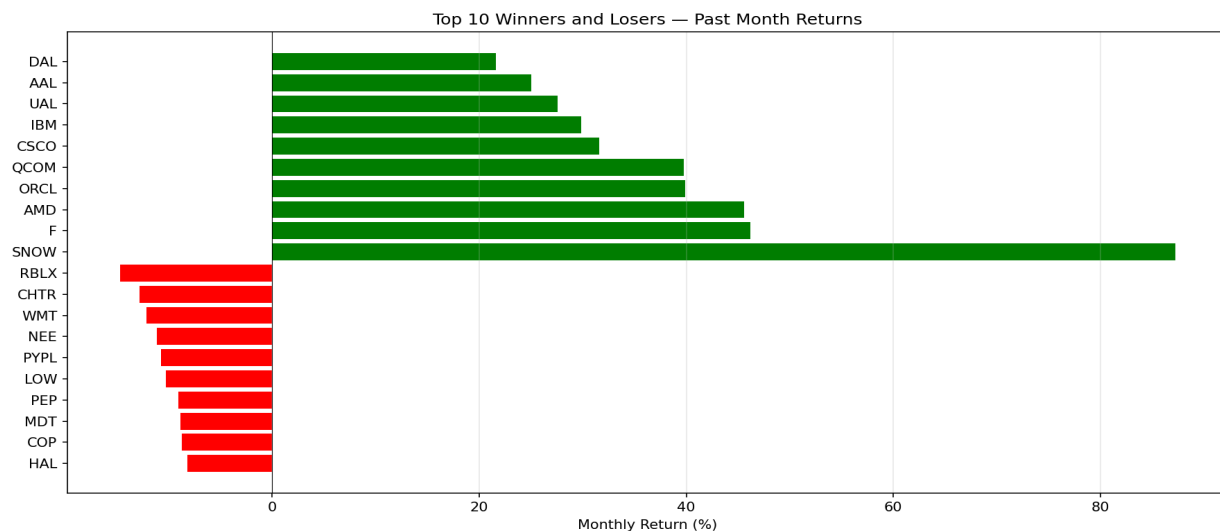


Figure 1 shows the top 10 winners and bottom 10 losers ranked by their past month's return. They were calculated using an aggregation pipeline that extracted the first and last closing prices from each company's price_history array. We see that the results show a rotation into cyclical and technology names during May 2026. Snowflake (SNOW) led all stocks with an 87% gain which is nearly double the next best performer. AMD (+46%), Ford (+46%), Oracle (+40%), and Qualcomm (+40%) also had strong returns. On the other end, the worst performers were mostly defensive and value-oriented stocks like Halliburton, ConocoPhillips, Medtronic, PepsiCo, and Lowe's. This divergence between growth and value names can suggest a risk-on market environment where investors moved capital away from "safe" sectors and into higher-growth opportunities.

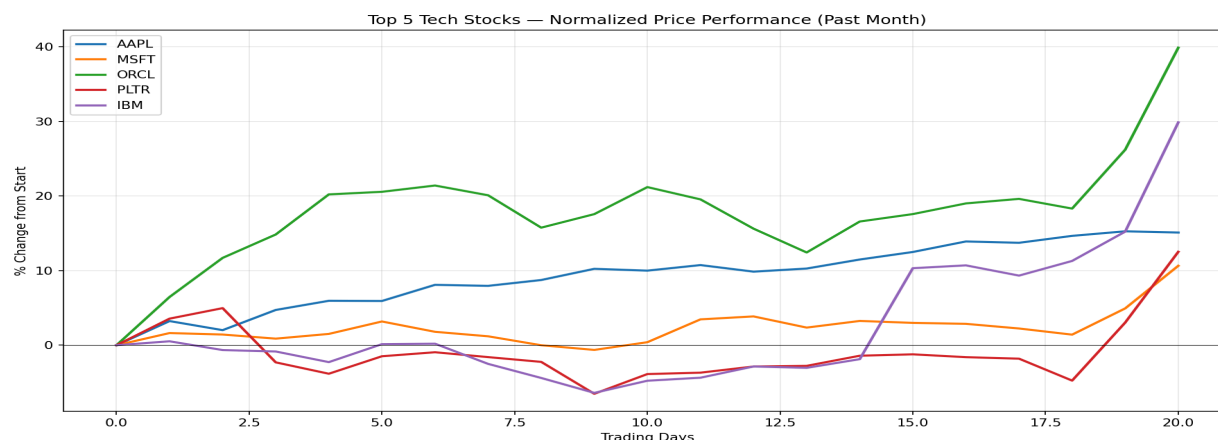


Figure 2 specifically looks into the Technology sector showing the normalized price performance of the five largest tech stocks by market cap over the past month. We see that Oracle led the group with about 40% gains, followed by IBM at around 30%, while Apple posted a steady 15% gain. Furthermore, Microsoft and Palantir were essentially flat for most of the month. This divergence within a single industry sector is the most important finding of this analysis. Oracle and IBM, which are more enterprise-focused, greatly outperformed consumer-facing names like Apple and Microsoft. A possible reason for this can be because enterprise IT spending and cloud infrastructure demand remained strong during this period.

Conclusions

We saw that MongoDB proved to be a good fit for this type of financial data analysis. The document model allowed me to store data from multiple API endpoints into a single collection without having to spend time defining a rigid schema upfront. Also the aggregation framework could compute derived metrics like monthly returns directly within the database, operations that would have required multiple JOIN statements and subqueries in a relational database. Overall, the key takeaway is that broad sector labels can be misleading because even within the Technology sector individual stock performance varied by over 40 percentage points in a single month. This reinforces the importance of a company level analysis over sector-level generalizations. For people trying to build a portfolio or just evaluating market trends, looking into the underlying company data instead of just relying on industry averages is important.

References

Finnhub API — <https://finnhub.io> (company profiles, financial metrics, earnings, peer data)

- I had a lot of finnhub api keys from a past project

Yahoo Finance via yfinance Python library — <https://pypi.org/project/yfinance/> (daily OHLCV price history)

Note

I think that the Yahoo Finance data that was fetched was not up to date maybe a couple weeks behind to actual market data but at the end of the day I am resorting to using free APIs/data sources

AI Usage

Throughout this homework I used the help of Claude to help pick out the stock symbols/tickers used in the Analysis because I wasn't going to handpick 50+ tickers by myself. I also used Claude come up with some of the questions about the data, as well as help with some of the more complicated syntax of pymongo when trying to query for data.